rmeshvk

A Differentiable Tetrahedral Volume Renderer

Technical Wiki

July 19, 2026

Contents

1 rmeshvk Architecture Overview **3**
- 1.1 What is rmeshvk? . . . 3
- 1.2 The core idea: sorting by circumsphere power . . . 3
- 1.3 The volume rendering integral . . . 3
- 1.4 Workspace crates . . . 4
- 1.5 The three rendering pathways . . . 4
- 1.6 Key design patterns . . . 5
- 1.7 Viewer frame graph . . . 5
- 1.8 The `.rmesh` data format . . . 5
- 1.9 Chapter guide . . . 6

2 The Interval Renderer (Main Pathway) **7**
- 2.1 Overview . . . 7
- 2.2 Stage 1 — Projection, culling, SH color . . . 7
- 2.3 Stage 2 — Radix sort . . . 8
- 2.4 Stage 3 — Indirect convert . . . 8
- 2.5 Stage 4 — Interval decomposition (`interval_compute.wgsl`) . . . 8
- 2.6 Stage 5 — Vertex shader (`interval_vertex.wgsl`) . . . 9
- 2.7 Stage 6 — Fragment shader (`interval_fragment.wgsl`) . . . 9
- 2.8 The mesh-shader variant . . . 9
- 2.9 Record functions and orchestration . . . 10

3 Mesh-Style Renderers (Legacy Pathways) **11**
- 3.1 Overview . . . 11
- 3.2 Regular: 12 triangles per tet (`ForwardPipelines`) . . . 11
- 3.3 Quad: billboard per tet . . . 12
- 3.4 Mesh-shader forward (`MeshForwardPipelines`) . . . 12
- 3.5 Ray-trace compute (`rmesh-raytrace`) . . . 12
- 3.6 Retained but unwired . . . 13

4 The Tile Renderer (Trainable Pathway) **14**
- 4.1 Overview . . . 14
- 4.2 Forward stage 1 — interval generation . . . 14
- 4.3 Forward stage 2 — tiled rasterization . . . 15
- 4.4 Backward stage 1 — per-tile reverse traversal . . . 15
- 4.5 Backward stage 2 — chain back to parameters . . . 16
- 4.6 Symbolic derivation and validation . . . 16

5 Deferred Shading **17**
- 5.1 Overview . . . 17
- 5.2 G-buffer layout . . . 17

5.3 World-position reconstruction . . . 18
5.4 Lighting model . . . 18
5.5 Ambient, occlusion, and screen-space effects . . . 18
5.6 Outputs, tone mapping, debug modes . . . 19
5.7 The non-deferred path: the compositor . . . 19

6 Shadow Rendering: Moment-Based Deep Shadow Maps 20
6.1 Why deep shadow maps . . . 20
6.2 Representation . . . 20
6.3 Generating the moments . . . 20
6.4 Atlas and metadata . . . 21
6.5 Sampling at shading time . . . 21
6.6 Debugging and validation . . . 21

7 GPU Sorting 23
7.1 Why sorting is the linchpin . . . 23
7.2 Backend selection . . . 23
7.3 Basic backend: 5-pass, 4-bit radix . . . 24
7.4 DRS backend: 3-kernel, 8-bit radix . . . 24
7.5 CPU fallback . . . 24
7.6 Tile-pair sorting for the trainable path . . . 24

8 Training Loop and Python Integration 26
8.1 Overview . . . 26
8.2 The native loop (`rmesh-train`) . . . 26
8.2.1 Loss (`loss_compute.wgsl`) . . . 26
8.2.2 Adam (`adam_compute.wgsl`) . . . 27
8.2.3 Per-tet error statistics (`rmesh-error`) . . . 27
8.3 Python bindings (`rmesh-python`) . . . 27
8.3.1 `RMeshRenderer` . . . 27
8.3.2 `autograd.py` and the tensor boundary . . . 27
8.4 Validation . . . 28

Chapter 1

rmeshvk Architecture Overview

1.1 What is rmeshvk?

rmeshvk is a wgpu-based differentiable tetrahedral volume renderer for *Radiance Meshes*: Delaunay tetrahedral meshes carrying per-tet density, base color, linear color gradients, and spherical-harmonics (SH) appearance. It is the Rust/WebGPU port of the rendering and training backend of the parent `delaunay_rasterization` project, replacing its Slang/CUDA pipeline. Everything — forward rendering, backward (gradient) passes, loss, and the Adam optimizer — runs on the GPU through WGSL compute and render pipelines, and a PyO3 module exposes the renderer to Python/PyTorch.

1.2 The core idea: sorting by circumsphere power

Rendering a translucent tetrahedral volume requires compositing tets in depth order. For a *Delaunay* tetrahedralization, the back-to-front order from a fixed origin o is given by a per-tet scalar, the *power* of the origin with respect to the tet's circumsphere:

$$P(T) \;=\; \|C(T) - o\|^2 \;-\; r(T)^2,$$

where C and r are the circumsphere center and radius (see `RADIANCE_MESHES.md`; the proof is due to Karasick et al. 1997). Crucially the order depends only on the ray *origin*, not the view direction, so one radix sort serves any projection sharing that origin (pinhole, fisheye, cubemap faces, off-axis). In the shaders the key is computed in a cancellation-stable form and bit-inverted so that an ascending radix sort yields back-to-front order:

```
let depth_raw = dot(cam - v0, cam + v0 - 2.0 * center);
sort_keys[tet_id] = ~bitcast<u32>(clamp(depth_raw, -1e20, 1e20));
```

Per-view frustum culling is decoupled from sorting and happens downstream in the projection / interval-generation shaders. See *GPU Sorting* (Chapter 7) for the sort implementations.

1.3 The volume rendering integral

All renderers share one analytic segment integral. A ray crosses a tet over a chord of length d; with density ρ, the optical depth is $\tau = \rho\, d$ (clamped to $[0, 88]$). Color varies linearly along the ray, so the exact integral has closed form:

$$\alpha = 1 - e^{-\tau}, \qquad \varphi(\tau) = \frac{1 - e^{-\tau}}{\tau}, \qquad w_0 = \varphi - e^{-\tau}, \qquad w_1 = 1 - \varphi,$$

$$c_{\text{premul}} = w_0\, c_{\text{back}} + w_1\, c_{\text{front}},$$

where $c_{\text{front}}, c_{\text{back}}$ are the colors at the entry and exit points ($c(p) = c_{\text{base}} + \nabla c \cdot (p - o_{\text{cam}})$, clamped non-negative). φ uses a 4-term Taylor expansion for $\tau < 0.02$ to avoid catastrophic cancellation (`crates/rmesh-util/src/wgsl/math.wgsl`). Compositing uses premultiplied alpha; the expected termination depth $w_0 z_{\text{back}} + w_1 z_{\text{front}}$ feeds deferred shading and shadow moments. The math is derived and finite-difference-validated by `scripts/derive_backward.py` (SymPy).

1.4 Workspace crates

Crate	Role
`rmesh-util`	Shared CPU/GPU `##[repr(C)]` structs; WGSL utility modules (math, intersect, SH)
`rmesh-sort`	GPU radix sort (Basic 5-pass, DRS subgroup) + CPU fallback sorter
`rmesh-tile`	Tile–tet pair generation, RTS prefix scan, tile ranges (training path)
`rmesh-data`	`.rmesh` loading, PCA SH decompression, circumsphere computation, GPU buffers
`rmesh-render`	Interval renderer (main) + legacy mesh-style renderers
`rmesh-raytrace`	Compute ray tracer with BVH entry + adjacency walk
`rmesh-trainable`	Differentiable tiled forward + backward (interval and baseline rasterizer)
`rmesh-error`	Per-tet error statistics for densification
`rmesh-train`	GPU-only training loop: loss, backward, Adam
`rmesh-dsm`	Deep shadow maps (2-moment, per-light cubemap atlas)
`rmesh-compositor`	Opaque primitive rendering (glTF materials) + depth compositing
`rmesh-postprocess`	Deferred shading, GTAO, SSGI, SSR, Hi-Z, blit
`rmesh-pbd`	GPU XPBD distance-constraint solver for vertex grabs
`rmesh-anim`	Animation clock, keyframes, glTF scene loader
`rmesh-interact`	Input handling, transform manipulators, vertex picking
`rmesh-viewer`	Interactive winit/wgpu viewer (orbit camera, egui UI)
`rmesh-viewer-web`	WebAssembly viewer (WebGPU limits, CPU sort)
`rmesh-python`	PyO3/maturin bindings (`RMeshRenderer`) for PyTorch

1.5 The three rendering pathways

Pathway	Crate	Purpose
Interval shader	`rmesh-render`	**Main** — viewer and forward inference
Mesh-style shaders	`rmesh-render`	**Legacy** — reference paths, viewer toggles
Tile shader	`rmesh-trainable`	**Trainable** — differentiable forward + backward

- *Interval shader* (Chapter 2) decomposes each tet into non-overlapping screen-space triangles carrying interpolated front/back depths, so the hardware rasterizer resolves overdraw with plain premultiplied blending — no per-fragment sorting.
- *Mesh-style shaders* (Chapter 3) are the original 12-triangle, quad-billboard, mesh-shader, and ray-trace paths, kept for reference and diagnostics.

- *Tile shader* (Chapter 4) is pure compute in both directions: a tiled front-to-back forward pass whose intermediates are exactly what the reverse-order backward pass needs.

On top of the volume renderers sit *deferred shading* (Chapter 5) — a G-buffer + fullscreen PBR pass with GTAO/SSGI/SSR — and *deep shadow maps* (Chapter 6). Training and the PyTorch bridge are covered in Chapter 8.

1.6 Key design patterns

- **Shared types** — `crates/rmesh-util/src/shared.rs` defines `##[repr(C)]` + `bytemuck::Pod` structs matching WGSL std430 layouts byte-for-byte (`Uniforms` is 192 bytes and is bound as a *storage* buffer in most passes to avoid std140 padding). All CPU↔GPU data flows through these.
- **Shaders as `include_str!`** — WGSL is embedded at compile time; `naga_oil` `##import` provides shared modules (`rmesh::math`, `rmesh::intersect`, `rmesh::sh`).
- **WebGPU limits** — bind groups are split in two to respect the 8–10 storage-buffer caps; the web viewer falls back to CPU sorting because WebGPU (wgpu 28) exposes no subgroup features.
- **Indirect everything** — projection compacts visible tets with an atomic counter; tiny "indirect convert" kernels turn that count into dispatch and draw arguments, so no CPU readback is needed per frame.
- **GPU-only training** — `rmesh-train` records forward + loss + backward + Adam into a single command encoder per view; the only readback is the scalar loss.

1.7 Viewer frame graph

Per frame, `rmesh-viewer/src/render.rs` records:

1. egui UI build; optional PBD solver step (vertex grabs).
2. Opaque primitive pass (glTF materials) — writes depth, and the full G-buffer when deferred shading is on.
3. Volume forward pass by `RenderMode`: `IntervalShader` (default), `Regular`, `Quad`, `MeshShader`, or `RayTrace`; all load (not clear) the primitive targets so the volume composites over the opaques.
4. Optional DSM regeneration (per light, per cubemap face) when lights moved or a recompute is forced.
5. Screen-space stack: Hi-Z pyramid → GTAO → SSGI → SSR (each with temporal accumulation) → deferred shade → sRGB blit to the swapchain.

1.8 The `.rmesh` data format

Gzip-compressed binary (`crates/rmesh-data`): header `[vertex_count, tet_count, sh_degree, k_components]` (u32); start pose (8× f32); vertices $[N \times 3]$ f32; indices $[M \times 4]$ u32; densities $[M]$ u8, log-encoded and decoded as $\exp((v - 100)/20)$; SH coefficients PCA-compressed (mean + basis + weights in f16) and decompressed on load as $\text{sh}[t][f] = \text{mean}[f] + \sum_k w[t][k]\, B[k][f]$; color gradients $[M \times 3]$ f16. Circumspheres are computed on load in double precision with degenerate-tet fallbacks (`compute_circumspheres_parallel`).

1.9 Chapter guide

- Chapter 2 — the main interval-shader renderer
- Chapter 3 — legacy renderers (Regular, Quad, MeshShader, RayTrace)
- Chapter 4 — the trainable tiled renderer and backward pass
- Chapter 5 — G-buffer, PBR lighting, screen-space effects
- Chapter 6 — moment-based deep shadow maps
- Chapter 7 — power sort theory and the radix sort backends
- Chapter 8 — training loop and PyTorch integration

Chapter 2

The Interval Renderer (Main Pathway)

2.1 Overview

The interval shader is the production renderer used by the interactive viewer (default `RenderMode::IntervalShader`) and by forward-only inference. Its central trick: instead of asking the fragment shader to figure out where a ray enters and exits a tetrahedron (which requires per-fragment plane intersections, as the legacy renderers do), a compute pass *decomposes each tet's screen projection into non-overlapping triangles* whose vertices carry the front and back NDC depths of the tet along that pixel's ray. The hardware rasterizer then interpolates the depth interval per pixel, and each covered pixel is shaded exactly once per tet with the correct entry/exit segment. Overdraw is resolved by ordinary premultiplied-alpha blending over back-to-front sorted tets — no fragment sorting, no fixed-function tricks.

Code lives in `crates/rmesh-render` (`src/lib.rs` and `src/wgsl/`). Two pipeline structs drive the path:

- `ComputeIntervalPipelines` — compute generation followed by a vertex/fragment draw. **Universal**: works on Metal, Vulkan, DX12 and WebGPU. This is the production path.
- `IntervalPipelines` — a single mesh-shader dispatch (`interval_mesh.wgsl`) replacing the generation + vertex stages. Requires `Features::EXPERIMENTAL_MESH_SHADER`; currently untested for lack of hardware.

2.2 Stage 1 — Projection, culling, SH color

`project_compute.wgsl` (or the lean `project_compute_hw.wgsl` when tile counts are not needed) runs one thread per tet (`@workgroup_size(64)`, 2D-dispatched above 65 535 workgroups):

1. Project the 4 vertices to NDC. If any `clip.w <= 0` the tet straddles the camera plane; it is kept and near-clipping is deferred to the rasterizer. Otherwise reject if the NDC AABB is fully outside $[-1, 1]^3$ or the screen extent is under one pixel^2.
2. Cull by maximum optical depth: skip if $\rho \cdot \max_{\text{edge}} < 0.01$.
3. Compute the depth sort key from the circumsphere power (see Chapter 7), bit-inverted so ascending order is back-to-front. Culled threads write `0xFFFFFFFF`, sorting to the end.
4. An `atomicAdd` on the instance count in `indirect_args` compacts the visible tets into `compact_tet_ids`.

5. Evaluate SH color (degree 0–3, f16 coefficients unpacked with `unpack2x16float`) in the direction $\widehat{\text{centroid} - \text{cam}}$, add the +0.5 DC bias and a gradient offset, then `softplus`; write to `colors[tet_id]`. Without SH data (training), base colors are copied through.

`project_compute_hw.wgsl` additionally binds `Uniforms` as a *uniform* buffer (dropping one storage binding for WebGPU's cap) and skips the tile-touch counting loop, which dominates the cost of the full variant. `project_compute_16bit.wgsl` quantizes depth to 16 bits for the faster `SortMode::Gpu16` (4 radix passes instead of 8).

2.3 Stage 2 — Radix sort

`rmesh-sort` sorts `(sort_keys, sort_values)` by depth. The DRS subgroup backend is used when `Features::SUBGROUP` is available, otherwise the Basic 5-pass backend; on backends without either (Chrome WebGPU) or with `SortMode::Cpu`, a CPU sorter does frustum culling and power sorting and its output is uploaded with a `StagingBelt` between the projection and generation submits. Details in Chapter 7. `record_radix_sort` returns `result_in_b`, which selects the A/B bind group downstream (the sort ping-pongs buffers).

2.4 Stage 3 — Indirect convert

`interval_indirect_convert.wgsl` runs a single thread that reads the visible count and fills an 8×u32 buffer `interval_args_buf`: words [0..2] are compute dispatch args $(\lceil n/64 \rceil, 1, 1)$ for the generation pass, words [3..7] are `draw_indexed_indirect` args $(12n, 1, 0, 0, 0)$ — twelve indices per tet. No CPU readback of the count is ever needed.

2.5 Stage 4 — Interval decomposition (`interval_compute.wgsl`)

One thread per sorted visible tet. The screen projection of a tetrahedron has exactly two silhouette topologies, and both are handled with a small, fixed-size output:

Case 1 — one vertex projects inside the opposite face. `classify_silhouette` tests each vertex against the opposite face's screen triangle with signed 2D cross products. The silhouette is that face's triangle; the interior vertex splits it into a 3-triangle fan around the *center* point. At the center, the ray passes through both the face (depth z_{face}, barycentric-interpolated) and the interior vertex (depth z_{center}); $z_{\text{front}} = \min$, $z_{\text{back}} = \max$. Emits 4 vertices, 3 triangles.

Case 2 — no interior vertex. Two opposite edges cross in screen space. `find_crossing_edges` tests the three opposite-edge pairs $(01, 23), (02, 13), (03, 12)$ with a 2D line–line intersection requiring both parameters in $(0, 1)$. The silhouette is a quad of the 4 vertices, split into a 4-triangle fan around the crossing point; the two crossing edges provide the front and back depths there. Emits 5 vertices, 4 triangles.

Per-vertex color offsets $\nabla c \cdot (v - o_{\text{cam}})$ are carried separately for the front and back surfaces, and the scalar-field gradient (used later as a normal) is carried for both. Degenerate tets (behind camera, depth range $< 10^{-7}$, density $< 10^{-6}$) write zeroed vertices.

Fixed-slot output. Every tet owns exactly 5 vertex slots (4 `vec4` each, 320 bytes/tet) in `interval_vertex_buf` at base `5 * tid`, plus two `vec4` of flat data (`base_color`, `density`, `tet_id`) in `interval_tet_data_buf`:

```
slot + 0 : (ndc_x, ndc_y, z_front, z_back)
slot + 1 : (color_off_front, color_off_back, 0, 0)
```

```
slot + 2 : (gradient_front.xyz, 0)
slot + 3 : (gradient_back.xyz, 0)
```

Because the layout is fixed, no atomic compaction is needed: a *static fan index buffer* (`interval_fan_index_buf`) built once at load indexes the slots directly — per tet i with $b = 5i$, the triangles

$$(b,\ b{+}1,\ b{+}4), \quad (b{+}1,\ b{+}2,\ b{+}4), \quad (b{+}2,\ b{+}3,\ b{+}4), \quad (b{+}3,\ b,\ b{+}4).$$

Case-1 tets duplicate slot 0 into slot 3, making the fourth fan triangle degenerate.

2.6 Stage 5 — Vertex shader (`interval_vertex.wgsl`)

The pipeline binds *no* vertex buffers; the vertex shader derives `tet = vertex_index / 5` and reads the slots from storage. It forwards NDC position, the depth pair, the color-offset pair, flat density / base-color / tet-id, and the interpolated field gradient.

2.7 Stage 6 — Fragment shader (`interval_fragment.wgsl`)

Per pixel:

1. Linearize the interpolated NDC depths: $z = \dfrac{nf}{f - z_{\text{ndc}}(f - n)}$.
2. Reconstruct the ray scale from the intrinsics, $s = \|(\frac{x-c_x}{f_x}, \frac{y-c_y}{f_y}, 1)\|$, giving the chord length $d = |z_b - z_f|\, s$.
3. Entry/exit colors $c_{f/b} = \max(c_{\text{base}} + \text{off}_{f/b}, 0)$.
4. Apply the closed-form volume integral (see the overview article): $\tau = \rho d$, $\alpha = 1 - e^{-\tau}$, $c_{\text{premul}} = w_0 c_b + w_1 c_f$.

Two entry points share the shading core: `main_color_only` writes just the color target (lean forward path), and `main` writes the full MRT G-buffer for deferred shading (all channels premultiplied by α for hardware blending):

Location	Format	Contents
0	Rgba16Float	albedo $\cdot\, \alpha$, α
1	Rgba16Float	roughness $\cdot\, \alpha$, $f_0 \cdot \alpha$, metallic $\cdot\, \alpha$, α
2	Rgba8Unorm	bias-encoded field-gradient normal $\cdot\, \alpha$
3	Rgba16Float	$\alpha E[z]$, 0, $\alpha E[z^2]$, α (expected termination depth)

$E[z^2]$ is exact for the exponential termination distribution within the segment, with a Taylor correction $\tau/6 - \tau^2/12 + \tau^3/40$ to avoid cancellation. The blend state is premultiplied over $(\text{src} \cdot 1 + \text{dst} \cdot (1 - \alpha_{\text{src}}))$, depth test `LessEqual` against the primitive depth with depth writes disabled, and no face culling (the triangles are screen-aligned).

2.8 The mesh-shader variant

`interval_mesh.wgsl` fuses stages 4–5 into one mesh-shader dispatch: `@workgroup_size(16)`, one thread per tet, 16 tets per workgroup. Each thread performs the same silhouette classification, then a serial 16-element workgroup prefix sum compacts variable per-tet output (4 verts/3 prims or 5 verts/4 prims) into the workgroup's mesh output arrays (`MAX_VERTS=240`, `MAX_PRIMS=192`).

A hard-won implementation note from the source: divergent threads must *never* early-return in a mesh shader (it hangs); output is guarded with `if my_vert_count > 0u` instead. Dispatch args come from `indirect_convert.wgsl` with `TETS_PER_GROUP=16` and `draw_mesh_tasks_indirect`.

2.9 Record functions and orchestration

In `crates/rmesh-render/src/lib.rs`:

- `record_sorted_compute_interval_forward_pass` — the GPU-sort path: reset args → project → radix sort → indirect convert → generation compute (`dispatch_workgroups_indirect`) → MRT/color render pass (`draw_indexed_indirect` at byte offset 12).
- `record_compute_interval_project` / `record_compute_interval_after_sort` — the split used by the CPU sort path, with a `StagingBelt` write of sorted indices and the visible count in between.
- `record_sorted_interval_forward_pass` — the mesh-shader variant.

The viewer (`crates/rmesh-viewer/src/render.rs`) selects the pass by `RenderMode` and `SortMode`; all volume passes use `LoadOp::Load` to composite over the earlier opaque-primitive pass. The same generation + moment-render machinery is reused from each light's perspective to build deep shadow maps (Chapter 6).

Chapter 3

Mesh-Style Renderers (Legacy Pathways)

3.1 Overview

Before the interval shader, rmeshvk rendered tets the classical way: draw geometry that covers the tet's screen footprint and let the *fragment* shader intersect the pixel ray with the tet's four planes to find the entry/exit segment. Four such renderers remain in the tree, selectable in the viewer via `RenderMode`: `Regular`, `Quad`, `MeshShader`, and `RayTrace`. They are no longer the default and are not used by training or the Python forward path, but they are valuable as cross-checks (the test suite compares renderers against a CPU reference) and as diagnostics. All of them share the projection + radix-sort front end and the closed-form volume integral described in the overview article.

Dispatch (`crates/rmesh-viewer/src/render.rs`):

```
match render_mode {
  IntervalShader => interval path (see interval-renderer.pdf),
  MeshShader if supported => record_sorted_mesh_forward_pass,
  RayTrace => locate + project + raytrace compute + copy to texture,
  _ /* Regular | Quad */ => record_sorted_forward_pass(use_quad),
}
```

3.2 Regular: 12 triangles per tet (`ForwardPipelines`)

`forward_vertex.wgsl` + `forward_fragment.wgsl`, `record_sorted_forward_pass`. Each visible tet is drawn as an instance of 12 vertices (4 faces × 3), with `instance_index` indexing the sorted visible list.

Vertex. A constant `FACE_VERTEX_MAP` picks the face and corner. The shader computes per-face *inward* normals, the ray–plane numerators $n \cdot (v_a - o_{\text{cam}})$, and denominators $n \cdot \text{ray}$, where the ray direction is deliberately left *unnormalized* as $p_{\text{world}} - o_{\text{cam}}$ so that perspective-correct interpolation reproduces the exact per-pixel vector. Back-facing triangles (numerator ≤ 0) are collapsed to zero area — only front faces of each tet are rasterized, and each pixel inside the silhouette is shaded once per front face it covers.

Fragment. Normalizes the interpolated ray, classifies the four planes into entering/exiting, and takes $t_{\min} = \max(t_{\text{enter}})$, $t_{\max} = \min(t_{\text{exit}})$, $d = \max(t_{\max} - t_{\min}, 0)$; then runs the shared `compute_integral` with the exit color first (webrm convention). MRT outputs: premultiplied

color, an aux target $(t_{\min}, t_{\max}, \tau, d)$, entry normal, and premultiplied depth. Note this aux layout differs from the deferred-shading G-buffer — this path predates it.

3.3 Quad: billboard per tet

Same record function with `use_quad = true`. A compute prepass (one thread per visible tet, `forward_prepass_compute.wgsl`) does the per-tet work once instead of 12 times per vertex:

- computes inward face normals and plane numerators;
- projects the 4 vertices, sorts them by angle around the centroid with a 5-comparison sorting network to get the convex hull in CCW order;
- writes a flat 160-byte record per tet: world positions in triangle-strip order $[0, 1, 3, 2]$, the four $(\text{normal}, \text{numerator})$ pairs, and $(\text{base color}, \text{density})$, $(\nabla c, 0)$.

`forward_vertex_quad.wgsl` then emits a 4-vertex triangle strip per tet from the precomputed buffer, and `forward_fragment_quad.wgsl` performs the same 4-plane classification as the Regular fragment shader. It adds a fallback for behind-camera tets: if the interpolated ray degenerates (sentinel ≈ 0), the ray is reconstructed from the pixel coordinates and camera intrinsics. The quad covers the whole silhouette with one primitive, trading vertex-shader work for slightly looser coverage.

3.4 Mesh-shader forward (`MeshForwardPipelines`)

`forward_mesh.wgsl` (`enable wgpu_mesh_shader`) generates the per-tet geometry directly in a mesh shader: `@workgroup_size(32)` = 8 tets × 4 threads; thread k of a tet owns vertex k *and* face k. Vertex positions and face normals are exchanged between the 4 threads with `subgroupShuffle` — groups of 4 never straddle a subgroup boundary for any subgroup size that is a multiple of 4. Each workgroup emits 32 vertices / 32 primitives; front-facing faces become real triangles, back-facing ones degenerate. The fragment shader is the unmodified `forward_fragment.wgsl`. Dispatched via `indirect_convert.wgsl` (`TETS_PER_GROUP=8`) and `draw_mesh_tasks_indirect`. Requires `Features::EXPERIMENTAL_MESH_SHADER`.

3.5 Ray-trace compute (`rmesh-raytrace`)

`RenderMode::RayTrace` is a software ray tracer used as the viewer's diagnostic mode — it needs *no sorting* at all, at the price of $O(\text{depth complexity})$ work per ray.

Flow per frame: the CPU walks the mesh to keep track of which tet contains the camera (`find_containing_tet_walk`); `project_compute` runs only for its SH color evaluation; then `raytrace_compute.wgsl` (`@workgroup_size(8,8)`) runs one thread per pixel:

1. Derive the ray from intrinsics (or from ray buffers when `ray_mode == 1`).
2. Find the entry tet: if the camera is inside the mesh, start from `start_tet`; otherwise trace a BVH over the *boundary faces* (32-deep stack, Möller–Trumbore tests; leaves pack `tet_id << 2 | face`).
3. **Adjacency walk**: intersect the ray with the current tet's four inward planes (entry = max t over entering faces, exit = min t over exiting faces), accumulate $c \mathrel{+}= c_{\text{premul}} \cdot T_j$ with $T_j = e^{\log T}$, plus normal, depth, and entropy moments; step through the exit face via `tet_neighbors[cur*4 + exit_face]`.
4. Stop on leaving the mesh, on $\log T \le -5.54$ (transmittance below $1/255$), or after 512 steps.

The result is written to a storage buffer ($\alpha = 1 - T_{\text{final}}$ plus an aux image with normal, depth, and entropy), then copied to an `Rgba32Float` texture for blitting. `locate_compute.wgsl` is a GPU point-location helper. Because the walk visits tets in exact ray order, this renderer is the ground truth the rasterizing paths are compared against.

3.6 Retained but unwired

`project_compute_16bit.wgsl` (half-precision sort keys, used by `SortMode::Gpu16`) and `shadow_ray_gen.wgsl` (a pre-DSM shadow experiment) are kept in the tree; the latter is superseded by the deep shadow maps described in Chapter 6.

Chapter 4

The Tile Renderer (Trainable Pathway)

4.1 Overview

`rmesh-trainable` implements the differentiable forward and backward passes used by `rmesh-train` and the Python bindings. Both directions are *pure compute* — no fixed-function rasterizer — so the forward intermediates (per-tet interval geometry, per-pixel transmittance state) are exactly the quantities the backward kernel needs to replay. The screen is divided into tiles (size 12 in training); a sorted list of (tile, tet) pairs assigns each tile the tets that touch it in depth order (built by `rmesh-tile` + the radix sort, see Chapter 7), and one workgroup processes each tile.

Two forward implementations exist:

- **Interval tiled** (`interval_generate.wgsl` + `interval_tiled_rasterize.wgsl`) — the current path, mirroring the interactive interval renderer's geometry.
- **Software rasterizer baseline** (`rasterize_compute.wgsl`) — the pre-interval kernel that does ray–tet plane intersection in-shader; still used by `rmesh-train`'s loop and kept as a baseline.

4.2 Forward stage 1 — interval generation

`interval_generate.wgsl` (`IntervalGeneratePipeline`) runs one thread per visible tet, indexed through `compact_tet_ids`. It performs the same silhouette decomposition as the interactive renderer (Chapter 2): Case 1 (interior vertex → 3-triangle fan) or Case 2 (crossing edges → 4-triangle fan), writing 5 fixed vertex slots per tet. Differences from the interactive variant:

- vertices are written in *pixel* coordinates rather than NDC ($x_{\text{px}} = (x_{\text{ndc}} + 1)\frac{W}{2}$, $y_{\text{px}} = (1 - y_{\text{ndc}})\frac{H}{2}$), ready for tile scanline tests;
- per-vertex records carry $(x, y, z^{\text{ndc}}_{\text{front}}, z^{\text{ndc}}_{\text{back}})$, the front/back color offsets, and front/back per-vertex normals;
- `interval_meta[vis_idx]` packs the silhouette vertex count in the low 4 bits and the tet id above (`num_sil | (tet_id << 4)`);
- per-tet flat data `interval_tet_data[vis_idx]` = (base color, ρ) with base color = $c + \nabla c \cdot (o_{\text{cam}} - v_0)$.

Behind-camera, out-of-frustum, and sub-pixel tets write degenerate slots and `interval_meta = 0`.

4.3 Forward stage 2 — tiled rasterization

`interval_tiled_rasterize.wgsl` (`IntervalTiledRasterizePipeline`): one workgroup of 32 threads (one warp) per tile, dispatched `(num_tiles, 1, 1)`. Workgroup shared memory holds per-pixel state for up to 16×16 pixels: accumulated premultiplied color with *log-space transmittance* in `.w`, a normal+depth accumulator, and a 5-float Mip-NeRF-360 distortion-loss state.

The tile's tets are traversed **front to back** by walking its `tile_ranges` entry from the near end. Per tet:

1. Load the 5 silhouette vertices; lanes each own one pixel row and compute the scanline span $[x_l, x_r]$ of the silhouette polygon on that row, trimming pixels already saturated ($\log T < -13.8$, i.e. $T < 10^{-6}$); lane 0 builds a prefix sum of row widths.
2. All 32 lanes stride over the covered pixels. For each pixel: find the containing fan triangle, compute barycentrics, interpolate $z_f^{\text{ndc}}, z_b^{\text{ndc}}$ and the color offsets.
3. Linearize depths, apply the ray scale, and evaluate the shared volume integral: $\tau = \rho d$, φ, w_0, w_1, $c_{\text{premul}} = w_0 c_b + w_1 c_f$ (colors ReLU-clamped).
4. Composite front-to-back in log space: $T_j = e^{\log T}$; if $T_j \geq 10^{-6}$: color $+= c_{\text{premul}} T_j$; $\log T \mathrel{-}= \tau$. Normal/depth and distortion accumulators are updated with the same T_j, and optional per-tet auxiliary channels accumulate $T_j \, \alpha \, \text{aux}[tet]$.

Final per-pixel writes: `rendered_image` RGBA with $\alpha = 1 - T_{\text{final}}$, a 4-channel `xyzd_image` (normal + expected depth), the 5-channel distortion state, and aux channels.

The baseline `rasterize_compute.wgsl` (`RasterizeComputePipeline`) has the same tile/warp/scanline structure but intersects the pixel ray with the tet's four faces in-shader (slab method over the shared `FACES` winding table) instead of reading pre-decomposed intervals.

4.4 Backward stage 1 — per-tile reverse traversal

`backward_interval_tiled.wgsl` (`BackwardIntervalTiledPipeline`), one warp per tile, two bind groups (WebGPU 8-buffer limit): read-only inputs (loss gradients `dl_d_image`, `dl_d_xyzd`, `dl_d_distortion`, `dl_d_aux`, the forward outputs, and the interval buffers) and atomic accumulators (`d_interval_verts`, `d_interval_tet_data`, `d_aux_data`).

The tile is traversed **back to front** — the reverse of the forward order — and the forward per-pixel state is reconstructed by *undo* rather than storage: initialize from the final rendered pixel ($\log T = \log(1 - \alpha)$), and per tet recompute the forward quantities, then peel its contribution off:

$$\log T_{\text{prev}} = \min(\log T + \tau,\, 0), \qquad \text{color}_{\text{prev}} = \text{color} - c_{\text{premul}}\, T_{\text{prev}}.$$

The chain rule then flows $dL/d(\text{image})$ into $d\,c_{\text{premul}}$, $d\tau$ (via $dw_0/d\tau = \varphi' + e^{-\tau}$, $dw_1/d\tau = -\varphi'$), $d\rho$, dd, and $dz_{f/b}^{\text{ndc}}$ (through the depth linearization Jacobian $\partial z_{\text{view}}/\partial z_{\text{ndc}} = nf(f-n)/(f - z_{\text{ndc}}(f-n))^2$), respecting every forward clamp (optical-depth clamp, ReLU on colors, NDC range). Contributions from the expected-depth and distortion outputs are handled the same way (distortion undoes its moment accumulators). Pixel gradients are scattered to the three fan-triangle vertices by barycentric weight.

Accumulation strategy. Each lane keeps private accumulators; a butterfly `subgroupShuffleXor` reduction sums the 32 lanes and *only lane 0* flushes with `atomicAdd` — one atomic add per channel per tet per tile instead of per pixel. Per-vertex gradient records hold 10 floats $(dz_f, dz_b, d\text{off}_f, d\text{off}_b, dn_f, dn_b)$; per-tet records hold ($d$ base color, $d\rho$).

4.5 Backward stage 2 — chain back to parameters

`interval_chain_back.wgsl` (`IntervalChainBackPipeline`), one thread per visible tet, maps screen-space gradients back to mesh parameters. It reconstructs the same Case-1/Case-2 topology and applies:

- **density**: pass-through into `d_densities[tet]`;
- **base color**: into `d_base_colors[tet]`; since base $= c + \nabla c \cdot (o_{\text{cam}} - v_0)$, also $d\nabla c \mathrel{+}= (o_{\text{cam}} - v_0)\,\Sigma d_{\text{bc}}$ and $dv_0 \mathrel{-}= \nabla c\,\Sigma d_{\text{bc}}$;
- **NDC depth → vertices**: $\partial z_{\text{ndc}}/\partial v_{\text{world}} = (\text{VP}_{z\,\text{row}} - z_{\text{ndc}}\,\text{VP}_{w\,\text{row}})/w_{\text{clip}}$, scattered by the barycentric / edge-mix weights of the case;
- **color offsets → gradient and vertices**: from off $= \nabla c \cdot (v - o_{\text{cam}})$;
- **normals**: silhouette and center vertex normal gradients scattered into `d_vertex_normals`.

All writes are `atomicAdd` — per-vertex buffers are indexed by global vertex id, so vertices shared between tets accumulate correctly.

Parameters receiving gradients: vertex positions, per-tet densities, per-tet base colors (the SH DC term — the trainable path runs with `sh_degree = 0`), per-tet color gradients, per-vertex normals, and optional per-tet aux channels.

4.6 Symbolic derivation and validation

`scripts/derive_backward.py` derives every piece of this math in SymPy and validates it with finite differences: the segment integral and its derivatives, the log-space compositing undo, ray–plane intersection Jacobians in compact cross-product form ($\partial n/\partial v_a = [v_b - v_c]_\times$, etc.), the color chain with ReLU guards, and a full single-tet/single-pixel end-to-end chain. GPU-side finite-difference gradient checks live in `crates/rmesh-render/tests/gradient_test.rs` and `crates/rmesh-trainable/tests/kernel_tests.rs`.

How this pathway is driven per training step — tile-pair generation, sorting, loss, and Adam — is covered in Chapter 8.

Chapter 5

Deferred Shading

5.1 Overview

When PBR data is present, the viewer shades in two phases: the volume and primitive renderers write a *G-buffer* (multiple render targets), and a fullscreen pass (`deferred_shade_frag.wgsl`, `DeferredShadePipeline` in `crates/rmesh-postprocess/src/lib.rs`) combines it with analytic lights, deep-shadow-map transmittance, ambient occlusion, and screen-space GI/reflections, then tone-maps. Everything is linear `Rgba16Float` end to end; sRGB encoding happens only in the final `blit.wgsl` copy to the swapchain.

5.2 G-buffer layout

Both producers — the volume interval renderer (`interval_fragment.wgsl`, see Chapter 2) and the opaque primitive pass (`primitive_mrt.wgsl` in `rmesh-compositor`) — write the *same* four color attachments, plus the hardware depth buffer. All volume channels are premultiplied by α so hardware blending composites them; primitives write $\alpha = 1$.

Slot	Binding	Format	Contents
albedo	1	Rgba16Float	albedo $\cdot\, \alpha,\ \alpha$
aux0 (PBR)	2	Rgba16Float	$r\alpha,\ f_0\alpha,\ m\alpha,\ \alpha$
normals	3	Rgba8Unorm	$(\frac{n}{2}+\frac{1}{2})\alpha,\ \alpha$
depth	4	Rgba16Float	$\alpha E[z],\ \text{occl} \cdot \alpha,\ \alpha E[z^2],\ \alpha$
hw depth	6	Depth32Float	hardware z

Conventions worth knowing:

- **Metallic sits in `aux0.b`**, dielectric f_0 in `aux0.g` (glTF metallic-roughness order); primitives write 0 there, and the deferred pass falls back to $f_0 = 0.04$.
- **Normals** are the only non-float16 target: bias-encoded `Rgba8Unorm`. Volume normals are the interpolated *scalar-field gradient* (density gradient), not a surface normal; primitives compute a TBN-mapped mesh normal. The deferred pass un-premultiplies, un-biases, and *negates* — converting COLMAP (Y-down, Z-forward) source conventions to wgpu (Y-up, Z-back).
- **Depth appears three ways** in the system: hardware `Depth32Float`, the volume's premultiplied expected-termination depth (with $E[z^2]$ for variance), and the shadow maps' normalized $[0, 1]$ moment depth. They must not be conflated.

5.3 World-position reconstruction

The fullscreen pass (a single vertex-index triangle, `draw(0..3)`) reconstructs the shaded point per pixel by taking the *closer* of the volume's expected termination depth (slot 3, normalized by its own α) and the linearized hardware depth, converting back to NDC and multiplying by the inverse view-projection.

5.4 Lighting model

Lights are a read-only storage buffer of up to 16 `GpuLight` records (`rmesh-util/src/shared.rs`): position, type (0 point, 1 spot, 2 directional), color, intensity, pre-normalized direction, and inner/outer cone cosines. The viewer collects them each frame from point-light and spot-light primitives and from active *flare projectiles* (the flare gun's glTF model carries an offset light).

Shading is a metallic-roughness GGX BRDF:

- Fresnel: Schlick, $F = F_0 + (1 - F_0)(1 - \cos\theta)^5$, with $F_0 = (1 - m)\, f_0 + m\,\text{albedo}$.
- Distribution: GGX/Trowbridge-Reitz with $a = r^2$ (roughness clamped to $[0.08, 1]$), $D = \dfrac{a^2}{\pi\big((n \cdot h)^2(a^2 - 1) + 1\big)^2}$.
- Visibility: height-correlated Smith GGX.
- Diffuse: $k_d = (1 - F_{\text{view}})(1 - m)$ times albedo, no $1/\pi$ (matching the parent project's parametric relighting branch).

Per light: attenuation is $1/d^2$ for point/spot (smooth cone falloff between the cone cosines for spots) and 1 for directional; the contribution is scaled by $T \cdot \text{atten}$ where T is the deep shadow map transmittance (Chapter 6). Two optimizations land per pixel: an early `continue` when $n \cdot l \leq 0$ skips the BRDF *and* the cubemap shadow sample for back-facing lights, and $k_d \cdot \text{albedo}$ is folded once per pixel.

5.5 Ambient, occlusion, and screen-space effects

The indirect term combines:

- **GTAO** (`gtao.wgsl`, R8Unorm + temporal accumulation and bilateral blur), multiplied with the primitive occlusion channel and blended by an `ao_strength` uniform;
- **ambient specular** via the Lazarov/Karis analytic environment-BRDF fit (`env_brdf_approx`) — no BRDF LUT texture;
- **SSGI** (`ssgi_compute.wgsl`) — hemisphere rays marched against a Hi-Z min-pyramid (`hiz_linearize.wgsl` / `hiz_downsample.wgsl`, R32Float, fusing hardware and volume depth), sampling the *previous frame's linear, pre-tonemap lit output* for one bounce. Misses (off-screen or step-budget exhausted) return **black**: SSGI is a pure on-screen bounce term, and the environment contribution lives in the composite instead;
- **SSR** (`ssr_compute.wgsl`) — a mirror ray for smooth surfaces; the indirect specular blends SSR toward the hemispherical *environment* color by smoothness^2, and an SSR miss falls back to the environment sampled along the world reflection direction.

The composite treats visibility asymmetrically, on purpose: the SSGI bounce already carries screen-space visibility in its rays, so it is *not* multiplied by AO (that would double-count occlusion);

the environment hemisphere term *is* AO-occluded. Per pixel:

$$\begin{aligned}
\text{env_diffuse} &= \text{mix}(\text{ground},\ \text{sky},\ \tfrac{n_y+1}{2}) \\
\text{ambient_diffuse} &= (1-F_{\text{env}})(1-m)\,\text{albedo}\,\big(\text{ssgi_strength}\cdot\text{ssgi} + \text{env_diffuse}\cdot\text{ao_factor}\big) \\
\text{ambient_specular} &= F_{\text{env}}\cdot\text{mix}(\text{env_specular},\ \text{ssr},\ \text{smoothness}^2)
\end{aligned}$$

and both terms are added to `lit`. The `ambient` slider is vestigial as a shading input: the CPU folds it into `sky\_color/ground\_color` as a uniform grey lift before upload (`render.rs`), so the environment hemisphere is the single source of non-bounce ambient light.

Pass order per frame: Hi-Z → GTAO (+temporal, blur) → SSGI (+temporal, blur) → SSR (+temporal) → deferred shade → blit.

5.6 Outputs, tone mapping, debug modes

The pass writes **two** targets: `display` (what the user sees, subject to debug overrides) and `lit` (the *linear, pre-tonemap, pre-exposure* shaded color, clamped at zero and rotated into a history texture that feeds next frame's SSGI — debug modes never pollute the feedback loop, and the bounce integrates HDR radiance rather than tone-compressed values). Tone mapping is the ACES Narkowicz fit applied to lit × exposure inline in the shader for `display`; gamma is deferred to `blit.wgsl`'s exact piecewise sRGB curve.

Nineteen debug modes (`DBG_*`) can override `display`: albedo (raw/true), normals, roughness, packed PBR, depth, specular-only, diffuse-only, average shadow transmittance, cubemap-face color, plaster (albedo replaced by white), alpha, primitives-only, DSM moments, AO, SSGI, lit history, SSR, and depth standard deviation $\sigma = \sqrt{E[z^2] - E[z]^2}$ from the moment channels.

5.7 The non-deferred path: the compositor

When deferred shading is off, volumes and primitives render to separate color targets and `rmesh-compositor`'s `composite.wgsl` merges them per pixel: recover the volume's view depth by un-premultiplying its expected-depth channel, recover the primitive depth from the hardware buffer, then either pass the volume through (no primitive), output the primitive opaquely (primitive in front), or composite the premultiplied volume *over* the primitive: $c = c_{\text{tet}} + (1 - \alpha_{\text{tet}})\, c_{\text{prim}}$. This is a single-layer depth resolve — correct because the volume was already composited in sorted order upstream and primitives are opaque.

The opaque primitives themselves (cube, sphere, plane, cylinder, custom glTF meshes) are drawn by `PrimitivePipeline` (`primitive_pass.rs`) with full glTF metallic-roughness materials (base-color, metal-rough, normal, and occlusion textures via a `MaterialRegistry`), in either single-target mode or 4-target MRT mode matching the G-buffer above.

Chapter 6

Shadow Rendering: Moment-Based Deep Shadow Maps

6.1 Why deep shadow maps

A translucent tetrahedral volume cannot use a classic depth-test shadow map: a light ray is not blocked at a single depth, it is *attenuated continuously*. What a receiver needs is the transmittance function $T(z)$ along the light ray. `rmesh-dsm` stores a compact statistical summary of the absorption along each light ray — two α-weighted depth moments — in a per-light cubemap, and reconstructs a conservative $T(z)$ bound at shading time. The design is documented in `MSM.md`; an earlier 4-moment trigonometric variant (Peters–Klein style) was removed in favor of this cheaper 2-moment scheme.

6.2 Representation

One `Rgba16Float` cubemap per light (max 16 lights), each texel:

$$\big(\, E[\alpha z],\ E[\alpha z^2],\ 0,\ \alpha_{\text{total}} \,\big),$$

where z is view-space depth normalized to $[0, 1]$ over the light's $[\text{near}, \text{far}]$ range and $\alpha_{\text{total}} = 1 - T_{\text{total}}$ is the total absorption along the ray. At 256^3 this costs 3 MiB per light (versus 9 MiB for the removed 4-moment variant).

6.3 Generating the moments

`generate_dsm_for_lights` (`crates/rmesh-dsm/src/lib.rs`) runs per light, *per cubemap face* ($\times 6$):

1. **Build the face view-projection** (`build_light_vp`): point lights get a 90° perspective per face (using face direction/up pairs matched to WebGPU cube conventions, plus a clip-space Y flip); spot lights a perspective with FOV $= 2\arccos(\cos_{\text{outer}})$; directional lights an orthographic box.
2. **Opaque blocker pre-pass** (`DsmPrimitivePipeline`, `dsm_primitive.wgsl`): primitives write $(z, z^2, 0, 1)$ — a full-α absorber concentrated at their depth — with depth writes on, so tets behind opaque geometry are culled by the depth test.
3. **Project + sort**: the forward `project_compute_hw` pipeline projects all tets through the face VP and emits depth keys, then the radix sort orders them back-to-front *for that face*.

Each face gets its own project + sort because a single per-light radial sort is *not* a valid per-face compositing order off-axis — this correctness fix (commit `9207ea3`) costs 6× the sort work per light but eliminates wrong shadows.

4. **Interval generation + moment render** (`DsmPipeline`): the interval renderer's geometry pass runs from the light's viewpoint, and `dsm_moment_fragment.wgsl` converts each tet's depth interval into moment contributions, blended back-to-front with premultiplied alpha ($\text{src} \cdot 1 + \text{dst} \cdot (1 - \alpha_{\text{src}})$) into a shared 2D staging texture, which is then copied into the cube face layer.

Moment fragment math. For a tet crossing the light ray over $[z_a, z_b]$ with optical depth τ: $\alpha = 1 - e^{-\tau}$, and using the same termination-distribution weights as the color integral ($\varphi = (1 - e^{-\tau})/\tau$, $w_0 = \varphi - e^{-\tau}$, $w_1 = 1 - \varphi$):

$$\text{out} = \big(w_0 z_b + w_1 z_a,\ \ w_0 z_b^2 + w_1 z_a^2,\ \ 0,\ \ \alpha\big) \ = \ \big(\alpha E[z],\ \alpha E[z^2],\ 0,\ \alpha\big).$$

Regeneration is triggered when lights move or appear (e.g. a flying flare projectile), or on demand from the UI.

6.4 Atlas and metadata

`DsmAtlas` owns the per-light cubemaps, the staging textures, and a storage buffer of `ShadowLightMeta`: per light, the six per-face view-projection *W rows only* (enough to compute the receiver's light-space depth; storing full matrices cost 4× the bytes and forced materializing all six for dynamic indexing), plus near/far and the light type.

6.5 Sampling at shading time

The deferred pass (Chapter 5) calls `evaluate_transmittance(world_pos, light)` per light:

1. Select the cube face from the major axis of $d = p_{\text{world}} - p_{\text{light}}$ and sample the moments with a filtering cube sampler.
2. Receiver depth: $w_{\text{clip}} = \text{VP}_{w\,\text{row}} \cdot (p_{\text{world}}, 1)$, normalized $z = \frac{w_{\text{clip}} - n}{f - n} - \text{bias}$.
3. Un-premultiply: $\mu = \frac{E[\alpha z]}{\alpha}$, $\overline{z^2} = \frac{E[\alpha z^2]}{\alpha}$.
4. If $z \le \mu$, the receiver is in front of the mean absorber: $T = 1$. Otherwise apply the **Cantelli (one-sided Chebyshev) bound** — the same inequality behind Variance Shadow Maps:

$$\sigma^2 = \max\big(\overline{z^2} - \mu^2,\ \sigma^2_{\min}\big), \qquad T(z) \ \le \ p_{\max} = \frac{\sigma^2}{\sigma^2 + (z - \mu)^2}.$$

5. Reduce light bleeding by remapping $\text{linstep}(0.15, 1, p_{\max})$, and floor the result at $1 - \alpha_{\text{total}}$ (light that provably gets through).

Constants: depth bias 1.5×10^{-3}, minimum variance 4×10^{-4}, bleed-reduction threshold 0.15. The bound is exact only for a two-spike absorber; for smoother distributions it *over*-shadows, which errs on the visually safe side.

6.6 Debugging and validation

`DsmResolvePipeline` (`dsm_resolve.wgsl`) reconstructs $T(z)$ from a moment texture at a UI-chosen query depth — it must use identical constants to the deferred shader —

and `record_dsm_resolve_cubemap_cross` lays the six faces out as a 3×2 cross for inspection. The deferred pass also has dedicated debug modes for average transmittance, cube-face selection, and raw DSM moments. A CPU-reference comparison lives in `crates/rmesh-dsm/tests/probe_vs_cpu_test.rs`.

References: Donnelly & Lauritzen, *Variance Shadow Maps* (2006); Peters & Klein, *Moment Shadow Mapping* (2015); Lokovic & Veach, *Deep Shadow Maps* (2000).

Chapter 7

GPU Sorting

7.1 Why sorting is the linchpin

Every rasterizing pathway composites translucent tets with premultiplied alpha, which is only correct in depth order. rmeshvk's tets come from a Delaunay tetrahedralization, and `RADIANCE_MESHES.md` records the property everything rests on: the back-to-front order from an origin o is the descending order of the *circumsphere power*

$$P(T) = \|C(T) - o\|^2 - r(T)^2,$$

independent of view direction (Karasick et al. 1997; cf. Edelsbrunner 1989, Max 1990). One sort per origin therefore serves any projection from that origin — the six faces of a shadow cubemap being the notable *non*-example, since each face pass still needs its own *culling*, and compositing correctness per face requires sorting with that face's projection (see Chapter 6).

Key construction. Expanding P around a tet vertex v_0 gives the cancellation-stable form used in `project_compute.wgsl`:

$$\text{depth} = (o - v_0) \cdot (o + v_0 - 2C),$$

clamped to $\pm 10^{20}$, bitcast to `u32`, and bit-inverted so an *ascending* radix sort yields *back-to-front*. Culled slots get `0xFFFFFFFF` and sort to the end; the value array carries tet ids. Degenerate (near-planar) tets get $r^2 = 10^{20}$ at load time, sorting them last. A 16-bit linear quantization of the key (`project_compute_16bit.wgsl`, `SortMode::Gpu16`) halves the radix passes when precision allows.

7.2 Backend selection

`rmesh-sort` picks a backend at startup:

- **DRS** when `Features::SUBGROUP` is available (native Vulkan/Metal/DX12);
- **Basic** otherwise (portable, workgroup atomics only);
- **CPU** for `SortMode::Cpu` and the web viewer, where wgpu 28's WebGPU backend exposes no subgroups and the Basic sort is visibly out of order at multi-million tet counts.

`record_radix_sort` returns `result_in_b`: the sort ping-pongs between A/B buffer pairs, and an odd pass count leaves the result in B, so downstream passes carry two prebuilt bind groups and pick by this flag.

7.3 Basic backend: 5-pass, 4-bit radix

`crates/rmesh-sort/src/basic.rs`, with one shader per pass (`radix_sort_count.wgsl` through `radix_sort_scatter.wgsl`); 256-thread workgroups, 4 elements/thread (blocks of 1024), 16 bins. Per 4-bit digit (8 passes for 32-bit keys):

1. **count** — per-block digit histograms via workgroup atomics;
2. **reduce** — tree-reduce each bin's block counts into a spine;
3. **scan** — single-workgroup exclusive prefix sum of the spine;
4. **scan_add** — add the scanned spine back into the per-block counts, yielding global bin offsets;
5. **scatter** — a stable in-block 2-bit-at-a-time local sort, then scatter keys and values to their global positions.

A `KEY_STRIDE` substitution reuses the same shaders for 32-bit tet keys and 64-bit tile keys.

7.4 DRS backend: 3-kernel, 8-bit radix

`crates/rmesh-sort/src/drs.rs` ports b0nes164's *DeviceRadixSort* (shaders `drs_upsweep.wgsl`, `drs_scan.wgsl`, `drs_downsweep.wgsl`): radix 256, 15 keys/thread, 3840-element partitions, 4 passes for 32 bits. Upsweep builds per-block and global digit histograms; a 256-workgroup scan (one per digit) prefixes the spine; the downsweep ranks keys cooperatively with subgroup operations and scatters. The per-pass key count is copied from a GPU counter, so the visible-tet count never touches the CPU.

7.5 CPU fallback

`crates/rmesh-sort/src/cpu.rs` (`CpuSorter`) performs Gribb–Hartmann sphere-vs-frustum culling (wgpu's $[0, 1]$ NDC makes the near plane row r_2, not $r_3 + r_2$), computes the same power key with the bit-inversion trick, packs `(key << 32) | tet_id` into `u64`, and uses `sort_unstable` (pdqsort — in place, avoiding a $\sim$38 MB radix scratch). The viewer uploads the resulting compact index list and visible count with a `StagingBelt` between the projection and interval-generation submits.

7.6 Tile-pair sorting for the trainable path

Training needs each screen tile's tets in depth order, which is a sort of *(tile, tet)* pairs under a 64-bit key (tile id in the high half, depth in the low half). `rmesh-tile` builds the pairs without CPU readback (`record_scan_tile_pipeline`):

1. **tile_fill** — fill the key/value arrays with the `0xFFFFFFFF` sentinel;
2. `project_compute` counts, per visible tet, the tiles its silhouette touches (conservative scanline test) into `tiles_touched`;
3. **prepare_dispatch** — one thread converts the visible count into dispatch arguments for the following passes;
4. **RTS prefix scan** (`rts.wgsl`, a port of b0nes164's reduce-then-scan, subgroup-based, 256 threads $\times$ vec4$\times$4 elements) turns the per-tet tile counts into write offsets — no atomics;
5. **tile_gen_scan** — each tet re-runs the scanline fill and emits one (key, tet id) pair per covered tile; the last tet writes the total pair count;

6. **radix sort** the pairs (`KEY_STRIDE=2`; the bit count adapts to the number of tile bits);
7. **tile_ranges** — one thread per pair detects tile boundaries in the sorted array and writes `tile_ranges[tile] = [start, end)`.

The tiled rasterizer and backward kernels then walk `tile_ranges` front-to-back and back-to-front respectively (Chapter 4). The forward interval renderer sets `tile_size_u = 0` to skip the tile-touch counting entirely — it is the dominant cost of the full projection kernel and only training needs it.

Chapter 8

Training Loop and Python Integration

8.1 Overview

Two consumers drive the trainable pathway (Chapter 4): the native Rust training loop in `rmesh-train`, and the PyTorch bridge in `rmesh-python`. Both share the same principle: record forward, loss, backward, and the optimizer into as few command encoders as possible and keep data on the GPU — the native loop's only per-step readback is the scalar loss.

8.2 The native loop (`rmesh-train`)

`train()` builds every pipeline, buffer, and bind group once (including A/B variants for the radix-sort ping-pong), then per view records a *single* command encoder:

1. clear gradient, loss, tile-range, and image buffers;
2. `project_compute` — SH/color evaluation, culling, per-tet tile counts, compact visible ids;
3. tile-pair generation and 64-bit radix sort, then tile ranges (Chapter 7);
4. forward rasterization — the native loop uses the software-rasterizer baseline (tile size 12, `RasterizeComputePipeline`); the interval variant is exercised through the Python bindings;
5. loss pass; backward pass; three Adam dispatches (one per parameter group).

One `queue.submit` per view; the only `write_buffer`s are the camera uniforms and per-group Adam uniforms. Cameras come from `TrainingView` with the pinhole convention (x right, y down, z forward) and intrinsics derived from the FOV.

8.2.1 Loss (`loss_compute.wgsl`)

A 16×16 workgroup per pixel block compares the premultiplied rendered RGB against ground truth. `loss_type = 0` gives L1 ($\partial L/\partial c = \text{sign}(c-g)/n$), otherwise L2 ($2(c-g)/n$); the result is written per pixel to `dl_d_image` (alpha gradient zero). SSIM has plumbing (a `lambda_ssim` uniform and `ssim_err` buffers consumed by `rmesh-error`) but is *not* computed by this shader — only L1/L2 are.

8.2.2 Adam (`adam_compute.wgsl`)

One dispatch per parameter group so each group gets its own learning rate: vertices (10^{-4}), densities (10^{-2}), color gradients (10^{-3}); $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. The kernel is a standard Adam step with NaN-guarded gradients and bias correction via $\beta^t = e^{t \ln \beta}$, using the safe-math helpers throughout. Moment buffers (m, v) live per group in `AdamState` / `MaterialAdamState`.

8.2.3 Per-tet error statistics (`rmesh-error`)

`error_tiled_compute.wgsl` re-traverses tiles exactly like the forward rasterizer but accumulates per-tet statistics with atomics, weighted by $T \cdot \tau$: total weight, weighted L1 and L1^2, weighted SSIM error, weighted entry/exit points and centroid, a hit count, and a max-visibility score. These drive densification decisions (which tets to split or prune).

8.3 Python bindings (`rmesh-python`)

`maturin develop` builds the `_native` module; the `rmesh_wgpu` package wraps it.

8.3.1 `RMeshRenderer`

A single `##[pyclass]` owns the entire wgpu state. Key methods:

- constructor: `(vertices, indices, base_colors, densities, color_grads, circumdata, width, height, aux_dim=0)`;
- forward variants: `forward` (hardware-raster path), `forward_tiled` (software rasterizer), `forward_interval_tiled` (interval path — returns a dict with image, xyzd, distortion, and aux arrays), `forward_raytrace`, `forward_error`;
- backward variants: `backward` and `backward_interval_tiled`, the latter taking `dl_d_image` and optional `dl_d_xyzd` / `dl_d_distortion` / `dl_d_aux` — each returns a dict of flat `f32` gradient arrays (`d_vertices`, `d_densities`, `d_base_colors`, `d_color_grads`, `d_vertex_normals`, optionally `d_aux_data`);
- `train_step(...) -> f32` — the all-GPU step (forward + loss + backward + Adam) with only the scalar loss read back;
- `update_params` / `get_params` and setters for vertex normals, aux data, and `min_t`.

`backward_interval_tiled` internally re-runs tile generation and sorting for the given view, then the two backward stages (per-tile reverse traversal, then the per-tet chain back to parameters) before reading back the gradient buffers.

8.3.2 `autograd.py` and the tensor boundary

`RMeshForward(torch.autograd.Function)`:

- **forward**: syncs the current PyTorch parameters into the GPU buffers via `update_params`, calls the chosen forward, and wraps the returned image back into a torch tensor on the caller's device;
- **backward**: moves `grad_output` to CPU numpy, calls the matching native backward, and wraps each gradient array as a torch tensor aligned with the forward argument tuple.

The boundary is *explicit host copies* in both directions (`.detach().cpu().numpy()` up, `read_buffer_f32` → numpy → `torch.from_numpy` down); there is no zero-copy CUDA↔wgpu interop. `RMeshModule(nn.Module)` exposes `vertices`, `base_colors`, `densities`, and

`color_grads` as `nn.Parameter`s so a standard `torch.optim.Adam(model.parameters())` loop trains the mesh — the alternative to the fully-native `train_step`.

8.4 Validation

Finite-difference gradient checks live in `crates/rmesh-render/tests/gradient_test.rs` and `crates/rmesh-trainable/tests/kernel_tests.rs`; the analytic math they check is derived symbolically in `scripts/derive_backward.py` (see Chapter 4). Renderer-vs-CPU-reference image comparisons live in `crates/rmesh-render/tests/`.